

ENSAH

DÉPARTEMENT INFORMATIQUE

Rapport de Projet

Gestion des Produits

Architecture MVC2 avec Hibernate

JEE · ORM · DAO

Présenté par : Lamhandi Taha
Encadré par : Mr, Cherradi
Module : Programmation Web JEE
Technologies : Java, Hibernate, JSP/Servlets

Année Universitaire 2025 – 2026

Table des matières

1	Introduction Générale	2
1.1	Contexte du Projet	2
1.2	Objectifs du Projet	2
1.3	Technologies Utilisées	3
2	Architecture MVC2	4
2.1	Principe du Pattern MVC2	4
2.1.1	Différences MVC1 vs MVC2	4
2.2	Diagramme d'Architecture	5
3	Configuration Hibernate	6
3.1	Fichier de Configuration	6
3.2	Classe Utilitaire Hibernate	7
3.3	Fichiers de Mapping	7
4	Modèles et Couche DAO	9
4.1	Classes Modèles	9
4.2	Interfaces DAO	10
4.3	Implémentations DAO	11
5	Couche Métier	13
5.1	Interfaces Métier	13
5.2	Implémentations Métier	14
6	Couche Web - Servlets	15
6.1	FrontController	15
6.2	Servlets d'Authentification	16
6.3	Servlets de Gestion des Produits	18
6.4	Filtre d'Authentification	21
7	Configuration de Déploiement	22
7.1	Fichier web.xml	22
7.2	Fichier pom.xml	23
8	Conclusion	24
8.1	Bilan du Projet	24

1 Introduction Générale

1.1 Contexte du Projet

Contexte

Dans le cadre de notre formation en développement d'applications web, nous avons été amenés à concevoir et réaliser une application de **gestion des produits** utilisant les technologies Java Entreprise Edition (JEE).

Ce projet vise à mettre en pratique les concepts fondamentaux du développement web côté serveur, notamment :

- Le pattern architectural **MVC2** (Model-View-Controller 2)
- L'utilisation d'un **ORM** (Object-Relational Mapping) avec Hibernate
- La persistance des données via **PostgreSQL**
- La sécurisation par **authentification et autorisation**

1.2 Objectifs du Projet

2vertsurfacewhite	
vertprincipal	
CRUD	Créer, Lire, Modifier et Supprimer des produits
Authentification	Système de login/register avec rôles (ADMIN/USER)
Filtrage	Restriction d'accès aux fonctionnalités sensibles
ORM	Mapping objet-relationnel avec Hibernate
MVC2	Séparation claire des responsabilités

TABLE 1.1 – Objectifs fonctionnels du projet

1.3 Technologies Utilisées

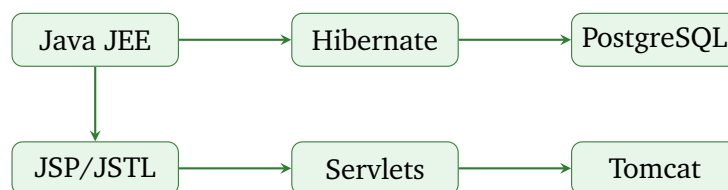


FIGURE 1.1 – Stack technique du projet

2 Architecture MVC2

2.1 Principe du Pattern MVC2

Définition

Le pattern **MVC2** (Model-View-Controller 2) est une architecture web où un **contrôleur frontal** (Front Controller) reçoit toutes les requêtes et les dispatch vers les contrôleurs spécifiques.

2.1.1 Différences MVC1 vs MVC2

2vertsurfacewhite		
vertprincipal		
Contrôleur	Un par vue	Front Controller unique
Routage	Direct dans la JSP	Via le Front Controller
URL	‘/login.jsp‘	‘/?action=login‘
Maintenance	Difficile	Centralisée

TABLE 2.1 – Comparaison MVC1 et MVC2

2.2 Diagramme d'Architecture

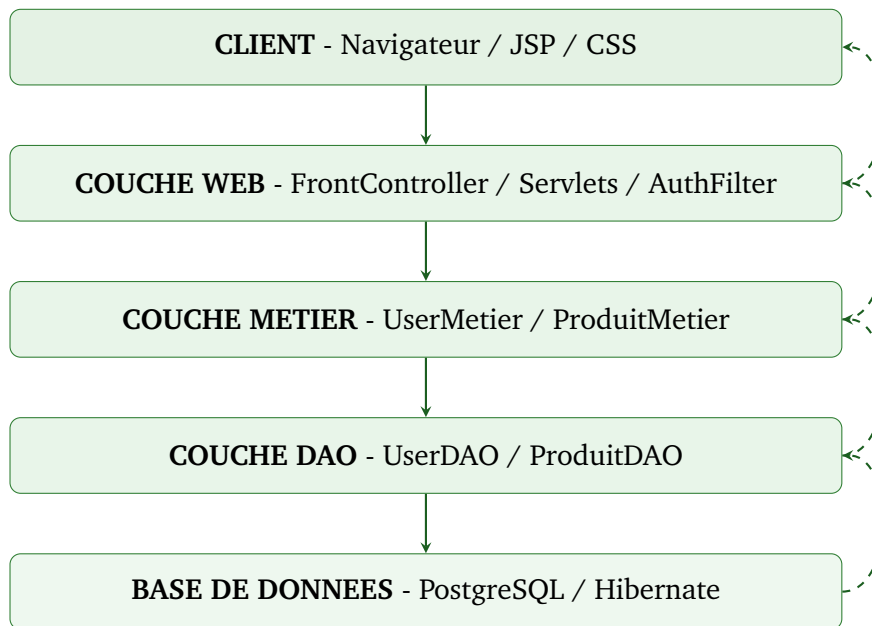


FIGURE 2.1 – Architecture en couches du projet

3 Configuration Hibernate

3.1 Fichier de Configuration

hibernate.cfg.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE hibernate-configuration PUBLIC
3     "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4     "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
5 <hibernate-configuration>
6     <session-factory>
7         <property name="hibernate.connection.driver_class">
8             org.postgresql.Driver
9         </property>
10        <property name="hibernate.connection.url">
11            jdbc:postgresql://localhost:5432/postgres
12        </property>
13        <property name="hibernate.connection.username">postgres</property>
14        <property name="hibernate.connection.password">102004</property>
15        <property name="hibernate.dialect">
16            org.hibernate.dialect.PostgreSQLDialect
17        </property>
18        <property name="hibernate.show_sql">true</property>
19        <property name="hibernate.hbm2ddl.auto">create</property>
20
21        <mapping resource="User.hbm.xml"/>
22        <mapping resource="Produit.hbm.xml"/>
23    </session-factory>
24 </hibernate-configuration>
```

3.2 Classe Utilitaire Hibernate

HibernateUtil.java

```
1 package utils;
2
3 import org.hibernate.SessionFactory;
4 import org.hibernate.cfg.Configuration;
5
6 public class HibernateUtil {
7     private static final SessionFactory sessionFactory;
8
9     static {
10         try {
11             sessionFactory = new Configuration()
12                 .configure("hibernate.cfg.xml")
13                 .buildSessionFactory();
14         } catch (Throwable ex) {
15             throw new ExceptionInInitializerError(ex);
16         }
17     }
18
19     public static SessionFactory getSessionFactory() {
20         return sessionFactory;
21     }
22
23     public static void shutdown() {
24         getSessionFactory().close();
25     }
26 }
```

3.3 Fichiers de Mapping

User.hbm.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE hibernate-mapping PUBLIC
3     "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4     "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
5
6 <hibernate-mapping>
7     <class name="dao.User" table="users">
8         <id name="id" column="id" type="java.lang.Long">
9             <generator class="identity"/>
10        </id>
11        <property name="name" column="name" type="string" length="50"
12            not-null="true"/>
13        <property name="email" column="email" type="string" length="100"
14            not-null="true" unique="true"/>
15        <property name="password" column="password" type="string" not-null="true"/>
16        <property name="role" column="role" type="string" not-null="true"/>
17    </class>
18 </hibernate-mapping>
```


Produit.hbm.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE hibernate-mapping PUBLIC
3     "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4     "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
5
6 <hibernate-mapping>
7     <class name="dao.Produit" table="produits">
8         <id name="id" column="id" type="java.lang.Long">
9             <generator class="identity"/>
10        </id>
11        <property name="nom" column="nom" type="string" length="100"
12            not-null="true"/>
13        <property name="description" column="description" type="string"
14            length="500"/>
15        <property name="prix" column="prix" type="double" not-null="true"/>
16    </class>
17 </hibernate-mapping>
```

4 Modèles et Couche DAO

4.1 Classes Modèles

User.java

```
1 package dao;
2
3 public class User {
4     private Long id;
5     private String name;
6     private String email;
7     private String password;
8     private String role;
9
10    public User() {}
11
12    public User(String name, String email, String password, String role) {
13        this.name = name;
14        this.email = email;
15        this.password = password;
16        this.role = role;
17    }
18
19    // Getters et Setters
20    public Long getId() { return id; }
21    public void setId(Long id) { this.id = id; }
22    public String getName() { return name; }
23    public void setName(String name) { this.name = name; }
24    public String getEmail() { return email; }
25    public void setEmail(String email) { this.email = email; }
26    public String getPassword() { return password; }
27    public void setPassword(String password) { this.password = password; }
28    public String getRole() { return role; }
29    public void setRole(String role) { this.role = role; }
30 }
```

Produit.java

```
1 package dao;
2
3 public class Produit {
4     private Long id;
5     private String nom;
6     private String description;
7     private Double prix;
8
9     public Produit() {}
10
11     public Produit(String nom, String description, Double prix) {
12         this.nom = nom;
13         this.description = description;
14         this.prix = prix;
15     }
16
17     // Getters et Setters
18     public Long getId() { return id; }
19     public void setId(Long id) { this.id = id; }
20     public String getNom() { return nom; }
21     public void setNom(String nom) { this.nom = nom; }
22     public String getDescription() { return description; }
23     public void setDescription(String description) { this.description =
24         description; }
25     public Double getPrix() { return prix; }
26     public void setPrix(Double prix) { this.prix = prix; }
27 }
```

4.2 Interfaces DAO

UserDAO.java

```
1 package dao;
2 import java.util.List;
3
4 public interface UserDAO {
5     User finduser(String email, String password);
6     boolean checkifemailexist(String email);
7     void save(User user);
8     List<User> findallusers();
9 }
```

ProduitDAO.java

```
1 package dao;
2 import java.util.List;
3
4 public interface ProduitDAO {
5     void addProduit(Produit p);
6     List<Produit> getAllProduits();
7     Produit getProduitById(int id);
8     void deleteProduit(int id);
9     void updateProduit(Produit p);
10 }
```

4.3 Implémentations DAO

UserImpl.java

```
1 package dao;
2 import utils.HibernateUtil;
3 import org.hibernate.Session;
4 import org.hibernate.Transaction;
5 import java.util.List;
6
7 public class UserImpl implements UserDAO {
8
9     @Override
10    public User finduser(String email, String password) {
11        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
12            return session.createQuery(
13                "from User u where u.email = :email and u.password = :password",
14                User.class)
15                .setParameter("email", email)
16                .setParameter("password", password)
17                .uniqueResult();
18        }
19    }
20
21    @Override
22    public boolean checkifemailexist(String email) {
23        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
24            Long count = session.createQuery(
25                "select count(*) from User u where u.email = :email", Long.class)
26                .setParameter("email", email)
27                .uniqueResult();
28            return count > 0;
29        }
30    }
31
32    @Override
33    public void save(User user) {
34        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
35            Transaction tx = session.beginTransaction();
36            session.save(user);
37            tx.commit();
38        }
39    }
40
41    @Override
42    public List<User> findallusers() {
43        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
44            return session.createQuery("from User", User.class).list();
45        }
46    }
47 }
```

ProduitImpl.java

```
1 package dao;
2 import utils.HibernateUtil;
3 import org.hibernate.Session;
4 import org.hibernate.Transaction;
5 import java.util.List;
6
7 public class ProduitImpl implements ProduitDAO {
8
9     @Override
10    public void addProduit(Produit p) {
11        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
12            Transaction tx = session.beginTransaction();
13            session.save(p);
14            tx.commit();
15        }
16    }
17
18    @Override
19    public List<Produit> getAllProduits() {
20        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
21            return session.createQuery("from Produit", Produit.class).list();
22        }
23    }
24
25    @Override
26    public Produit getProduitById(int id) {
27        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
28            return session.get(Produit.class, (long) id);
29        }
30    }
31
32    @Override
33    public void deleteProduit(int id) {
34        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
35            Transaction tx = session.beginTransaction();
36            Produit p = session.get(Produit.class, (long) id);
37            if (p != null) session.delete(p);
38            tx.commit();
39        }
40    }
41
42    @Override
43    public void updateProduit(Produit p) {
44        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
45            Transaction tx = session.beginTransaction();
46            session.update(p);
47            tx.commit();
48        }
49    }
50 }
```

5 Couche Métier

5.1 Interfaces Métier

UserMetier.java

```
1 package metier;
2 import dao.User;
3
4 public interface UserMetier {
5     User login(String email, String password);
6     User register(String name, String email, String password);
7 }
```

ProduitMetier.java

```
1 package metier;
2 import dao.Produit;
3 import java.util.List;
4
5 public interface ProduitMetier {
6     void addProduit(Produit p);
7     void deleteProduit(int id);
8     List<Produit> getAllProduits();
9     Produit getProduitById(int id);
10    void updateProduit(Produit p);
11 }
```

5.2 Implémentations Métier

UserMetierImpl.java

```
1 package metier;
2 import dao.User;
3 import dao.UserDAO;
4 import dao.UserImpl;
5
6 public class UserMetierImpl implements UserMetier {
7     private UserDAO userDAO;
8
9     public UserMetierImpl() {
10         this.userDAO = new UserImpl();
11     }
12
13     @Override
14     public User login(String email, String password) {
15         return userDAO.finduser(email, password);
16     }
17
18     @Override
19     public User register(String name, String email, String password) {
20         if (!userDAO.checkifemailexist(email)) {
21             User newuser = new User(name, email, password, "USER");
22             userDAO.save(newuser);
23             return newuser;
24         }
25         return null;
26     }
27 }
```

ProduitMetierImpl.java

```
1 package metier;
2 import dao.Produit;
3 import dao.ProduitDAO;
4 import dao.ProduitImpl;
5 import java.util.List;
6
7 public class ProduitMetierImpl implements ProduitMetier {
8     private ProduitDAO produitDAO;
9
10    public ProduitMetierImpl() {
11        this.produitDAO = new ProduitImpl();
12    }
13
14    @Override
15    public void addProduit(Produit p) { produitDAO.addProduit(p); }
16
17    @Override
18    public void deleteProduit(int id) { produitDAO.deleteProduit(id); }
19
20    @Override
21    public List<Produit> getAllProduits() { return produitDAO.getAllProduits(); }
22
23    @Override
24    public Produit getProduitById(int id) { return produitDAO.getProduitById(id); }
25
26    @Override
27    public void updateProduit(Produit p) { produitDAO.updateProduit(p); }
28 }
```

6 Couche Web - Servlets

6.1 FrontController

FrontController.java

```
1 package web;
2 import utils.HibernateUtil;
3 import javax.servlet.ServletException;
4 import javax.servlet.http.HttpServlet;
5 import javax.servlet.http.HttpServletRequest;
6 import javax.servlet.http.HttpServletResponse;
7 import java.io.IOException;
8
9 public class FrontController extends HttpServlet {
10
11     @Override
12     public void init() throws ServletException {
13         HibernateUtil.getSessionFactory();
14     }
15
16     @Override
17     protected void doGet(HttpServletRequest req, HttpServletResponse res)
18         throws ServletException, IOException {
19         process(req, res);
20     }
21
22     @Override
23     protected void doPost(HttpServletRequest req, HttpServletResponse res)
24         throws ServletException, IOException {
25         process(req, res);
26     }
27
28     private void process(HttpServletRequest req, HttpServletResponse res)
29         throws ServletException, IOException {
30         String action = req.getParameter("action");
31         if (action == null) action = "login";
32
33         switch (action) {
34             case "login": new LoginServlet().doPost(req, res); break;
35             case "register": new RegisterServlet().doPost(req, res); break;
36             case "list": new ListProduitsServlet().doGet(req, res); break;
37             case "ajouter": new AjouterProduitServlet().doPost(req, res); break;
38             case "supprimer": new SupprimerProduitServlet().doGet(req, res); break;
39             case "modifier": new ModifierProduitServlet().doGet(req, res); break;
40             case "update": new UpdateProduitServlet().doPost(req, res); break;
41             case "logout": new LogoutServlet().doGet(req, res); break;
42             default: req.getRequestDispatcher("login.jsp").forward(req, res);
43         }
44     }
45 }
```


6.2 Servlets d'Authentification

LoginServlet.java

```
1 package web;
2 import dao.User;
3 import metier.UserMetier;
4 import metier.UserMetierImpl;
5 import javax.servlet.ServletException;
6 import javax.servlet.http.*;
7 import java.io.IOException;
8
9 public class LoginServlet extends HttpServlet {
10     private UserMetier userMetier;
11
12     @Override
13     public void init() throws ServletException {
14         userMetier = new UserMetierImpl();
15     }
16
17     @Override
18     protected void doPost(HttpServletRequest req, HttpServletResponse res)
19         throws IOException, ServletException {
20         String email = req.getParameter("email");
21         String password = req.getParameter("password");
22         HttpSession session = req.getSession();
23         session.removeAttribute("error");
24
25         User currentUser = userMetier.login(email, password);
26         if (currentUser != null) {
27             session.setAttribute("currentUser", currentUser);
28             res.sendRedirect("list");
29         } else {
30             session.setAttribute("error", "Email ou mot de passe incorrect");
31             req.getRequestDispatcher("login.jsp").forward(req, res);
32         }
33     }
34 }
```

RegisterServlet.java

```
1 package web;
2 import dao.User;
3 import metier.UserMetier;
4 import metier.UserMetierImpl;
5 import javax.servlet.ServletException;
6 import javax.servlet.http.*;
7 import java.io.IOException;
8
9 public class RegisterServlet extends HttpServlet {
10     private UserMetier metier;
11
12     @Override
13     public void init() throws ServletException {
14         this.metier = new UserMetierImpl();
15     }
16
17     @Override
18     protected void doPost(HttpServletRequest req, HttpServletResponse res)
19         throws ServletException, IOException {
20         String name = req.getParameter("name");
21         String email = req.getParameter("email");
22         String password = req.getParameter("password");
23         HttpSession session = req.getSession();
24
25         User newuser = metier.register(name, email, password);
26         if (newuser != null) {
27             session.setAttribute("currentUser", newuser);
28             res.sendRedirect("list");
29         } else {
30             session.setAttribute("error", "Email deja associe a un compte");
31             req.getRequestDispatcher("register.jsp").forward(req, res);
32         }
33     }
34 }
```

6.3 Servlets de Gestion des Produits

ListProduitsServlet.java

```
1 package web;
2 import dao.Produit;
3 import metier.ProduitMetier;
4 import metier.ProduitMetierImpl;
5 import javax.servlet.ServletException;
6 import javax.servlet.http.*;
7 import java.io.IOException;
8 import java.util.List;
9
10 public class ListProduitsServlet extends HttpServlet {
11     private ProduitMetier metier;
12
13     public void init() throws ServletException {
14         this.metier = new ProduitMetierImpl();
15     }
16
17     @Override
18     protected void doGet(HttpServletRequest req, HttpServletResponse res)
19         throws IOException, ServletException {
20         String id = req.getParameter("id");
21
22         if (id != null) {
23             Produit p = metier.getProduitById(Integer.parseInt(id));
24             if (p != null) {
25                 req.setAttribute("produits", List.of(p));
26             } else {
27                 req.setAttribute("error", "Cet ID n'existe pas");
28                 req.setAttribute("produits", metier.getAllProduits());
29             }
30         } else {
31             req.setAttribute("produits", metier.getAllProduits());
32         }
33
34         req.getRequestDispatcher("listproduits.jsp").forward(req, res);
35     }
36 }
```

AjouterProduitServlet.java

```
1 package web;
2 import dao.Produit;
3 import metier.ProduitMetier;
4 import metier.ProduitMetierImpl;
5 import javax.servlet.ServletException;
6 import javax.servlet.http.*;
7 import java.io.IOException;
8
9 public class AjouterProduitServlet extends HttpServlet {
10     private ProduitMetier metier;
11
12     @Override
13     public void init() throws ServletException {
14         this.metier = new ProduitMetierImpl();
15     }
16
17     @Override
18     protected void doPost(HttpServletRequest req, HttpServletResponse res)
19         throws IOException, ServletException {
20         String nom = req.getParameter("nomprod");
21         String description = req.getParameter("description");
22         Double prix = Double.parseDouble(req.getParameter("prix"));
23
24         metier.addProduit(new Produit(nom, description, prix));
25         res.sendRedirect("list");
26     }
27 }
```

SupprimerProduitServlet.java

```
1 package web;
2 import metier.ProduitMetier;
3 import metier.ProduitMetierImpl;
4 import javax.servlet.ServletException;
5 import javax.servlet.http.*;
6 import java.io.IOException;
7
8 public class SupprimerProduitServlet extends HttpServlet {
9     private ProduitMetier metier;
10
11     @Override
12     public void init() throws ServletException {
13         this.metier = new ProduitMetierImpl();
14     }
15
16     @Override
17     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
18         throws ServletException, IOException {
19         int id = Integer.parseInt(req.getParameter("id"));
20         metier.deleteProduit(id);
21         resp.sendRedirect("list");
22     }
23 }
```

ModifierProduitServlet.java

```
1 package web;
2 import dao.Produit;
3 import metier.ProduitMetier;
4 import metier.ProduitMetierImpl;
5 import javax.servlet.ServletException;
6 import javax.servlet.http.*;
7 import java.io.IOException;
8
9 public class ModifierProduitServlet extends HttpServlet {
10     private ProduitMetier metier;
11
12     @Override
13     public void init() throws ServletException {
14         this.metier = new ProduitMetierImpl();
15     }
16
17     @Override
18     protected void doGet(HttpServletRequest req, HttpServletResponse res)
19         throws ServletException, IOException {
20         int id = Integer.parseInt(req.getParameter("id"));
21         req.setAttribute("produitEdit", metier.getProduitById(id));
22         req.setAttribute("produits", metier.getAllProduits());
23         req.getRequestDispatcher("listproduits.jsp").forward(req, res);
24     }
25 }
```

UpdateProduitServlet.java

```
1 package web;
2 import dao.Produit;
3 import metier.ProduitMetier;
4 import metier.ProduitMetierImpl;
5 import javax.servlet.ServletException;
6 import javax.servlet.http.*;
7 import java.io.IOException;
8
9 public class UpdateProduitServlet extends HttpServlet {
10     private ProduitMetier metier;
11
12     @Override
13     public void init() throws ServletException {
14         this.metier = new ProduitMetierImpl();
15     }
16
17     @Override
18     protected void doPost(HttpServletRequest req, HttpServletResponse res)
19         throws ServletException, IOException {
20         Produit p = new Produit();
21         p.setId(Long.parseLong(req.getParameter("id")));
22         p.setNom(req.getParameter("nomprod"));
23         p.setDescription(req.getParameter("description"));
24         p.setPrix(Double.parseDouble(req.getParameter("prix")));
25
26         metier.updateProduit(p);
27         res.sendRedirect("list");
28     }
29 }
```

LogoutServlet.java

```
1 package web;
2 import javax.servlet.ServletException;
3 import javax.servlet.http.*;
4 import java.io.IOException;
5
6 public class LogoutServlet extends HttpServlet {
7     @Override
8     protected void doGet(HttpServletRequest req, HttpServletResponse res)
9         throws ServletException, IOException {
10         req.getSession().invalidate();
11         res.sendRedirect("login.jsp");
12     }
13 }
```

6.4 Filtre d'Authentification

AuthFilter.java

```
1 package web;
2 import javax.servlet.*;
3 import javax.servlet.http.*;
4 import java.io.IOException;
5
6 public class AuthFilter implements Filter {
7     @Override
8     public void doFilter(ServletRequest request, ServletResponse response,
9         FilterChain chain)
10         throws IOException, ServletException {
11         HttpServletRequest req = (HttpServletRequest) request;
12         HttpServletResponse res = (HttpServletResponse) response;
13
14         String action = req.getParameter("action");
15         if (action == null) action = "login";
16
17         if (action.equals("login") || action.equals("register")) {
18             chain.doFilter(req, res);
19             return;
20         }
21
22         HttpSession session = req.getSession(false);
23         if (session != null && session.getAttribute("currentUser") != null) {
24             chain.doFilter(req, res);
25         } else {
26             res.sendRedirect("login.jsp");
27         }
28     }
29 }
```

7 Configuration de Déploiement

7.1 Fichier web.xml

web.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
5                             http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
6         version="3.1">
7
8     <display-name>GestionDesProduitsMVC2</display-name>
9
10    <welcome-file-list>
11        <welcome-file>login.jsp</welcome-file>
12    </welcome-file-list>
13
14    <servlet>
15        <servlet-name>FrontController</servlet-name>
16        <servlet-class>web.FrontController</servlet-class>
17        <load-on-startup>1</load-on-startup>
18    </servlet>
19    <servlet-mapping>
20        <servlet-name>FrontController</servlet-name>
21        <url-pattern>/</url-pattern>
22    </servlet-mapping>
23
24    <filter>
25        <filter-name>AuthFilter</filter-name>
26        <filter-class>web.AuthFilter</filter-class>
27    </filter>
28    <filter-mapping>
29        <filter-name>AuthFilter</filter-name>
30        <url-pattern>*</url-pattern>
31    </filter-mapping>
32
33 </web-app>
```

7.2 Fichier pom.xml

pom.xml

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3       xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
4       http://maven.apache.org/xsd/maven-4.0.0.xsd">
5   <modelVersion>4.0.0</modelVersion>
6   <groupId>com.gestionproduits</groupId>
7   <artifactId>GestionDesProduitsMVC2</artifactId>
8   <version>1.0-SNAPSHOT</version>
9   <packaging>war</packaging>
10
11   <properties>
12     <maven.compiler.source>11</maven.compiler.source>
13     <maven.compiler.target>11</maven.compiler.target>
14   </properties>
15
16   <dependencies>
17     <dependency>
18       <groupId>org.hibernate</groupId>
19       <artifactId>hibernate-core</artifactId>
20       <version>5.4.33.Final</version>
21     </dependency>
22     <dependency>
23       <groupId>org.postgresql</groupId>
24       <artifactId>postgresql</artifactId>
25       <version>42.7.3</version>
26     </dependency>
27     <dependency>
28       <groupId>org.slf4j</groupId>
29       <artifactId>slf4j-simple</artifactId>
30       <version>1.7.36</version>
31     </dependency>
32     <dependency>
33       <groupId>javax.servlet</groupId>
34       <artifactId>javax.servlet-api</artifactId>
35       <version>4.0.1</version>
36       <scope>provided</scope>
37     </dependency>
38     <dependency>
39       <groupId>javax.servlet</groupId>
40       <artifactId>jstl</artifactId>
41       <version>1.2</version>
42     </dependency>
43   </dependencies>
44 </project>
```


8 Conclusion

8.1 Bilan du Projet

Réalisations

Ce projet nous a permis de maîtriser les concepts fondamentaux du développement web JEE :

- Architecture **MVC2** avec contrôleur frontal
- Persistance avec **Hibernate ORM** et mapping XML
- Requêtes **HQL** (Hibernate Query Language)
- Sécurisation par filtres et sessions
- Gestion des rôles (ADMIN/USER)

Fin du rapport

Merci pour votre attention